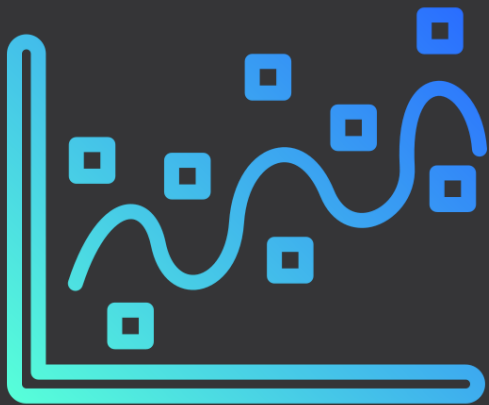




Regularization

The Problem of Overfitting

Lecture 11 – HCCDA-AI

Dr. Muhammad Sajjad

RA: Imran Nawar

Regularization: The Problem of Overfitting

Overfitting occurs when a model performs exceptionally well on the training data but poorly on new, unseen data.

Model Fitting Scenarios:

- **Overfitting:**

- The model learns noise and specific details of the training set, leading to poor generalization.

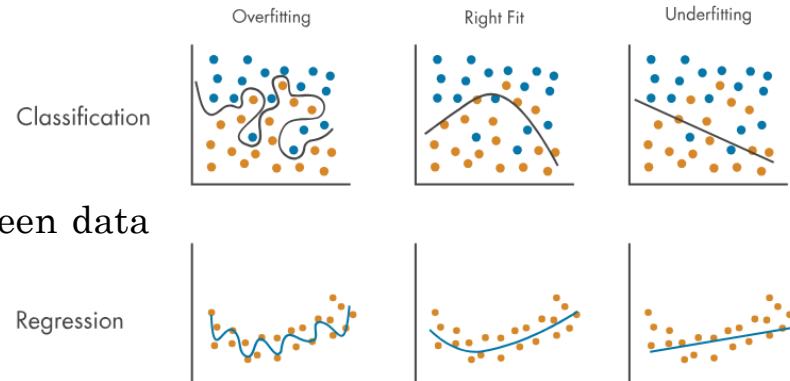
- **Just Right (Good Generalization):**

- The model captures patterns well and generalizes effectively to new data.

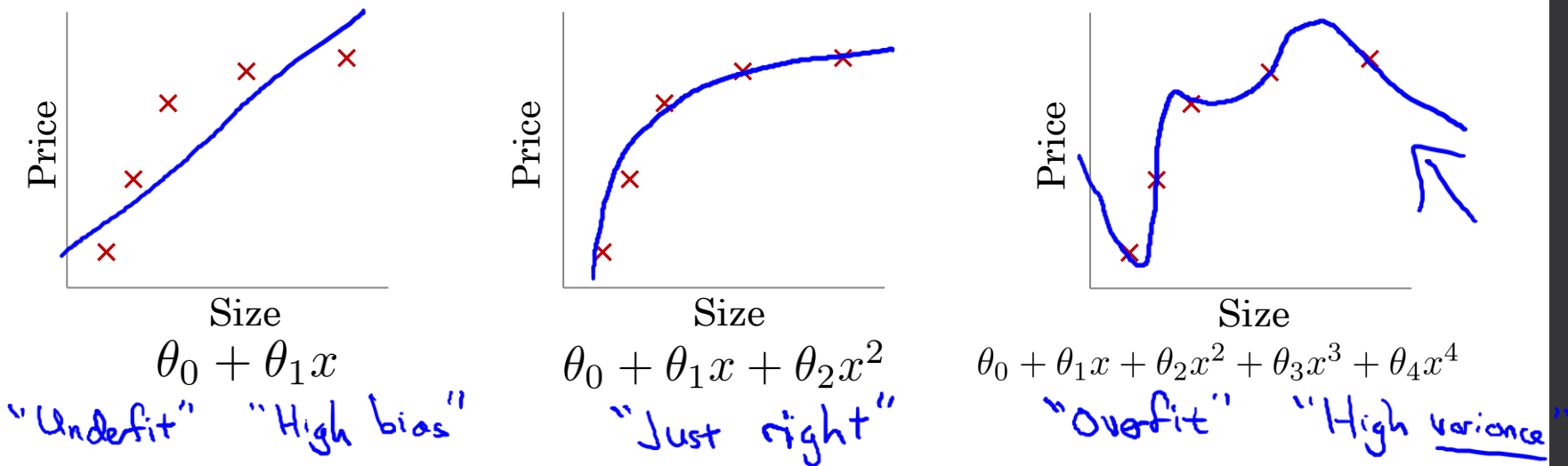
- **Underfitting:**

- The model does not capture patterns well, leading to poor performance on both training and validation data.

- **Goal:** Build a model that generalizes well to unseen data while avoiding overfitting.

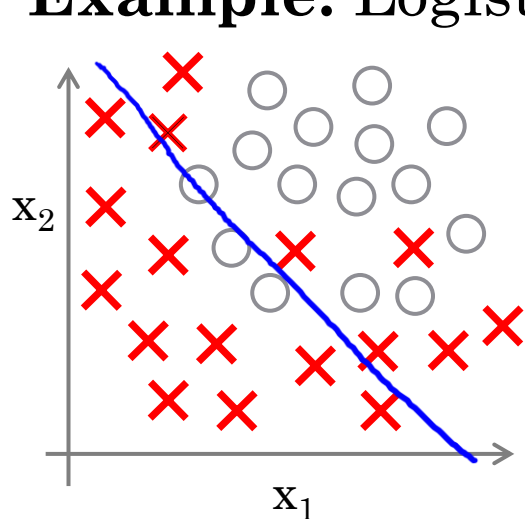


Example: Linear regression (housing prices)



Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

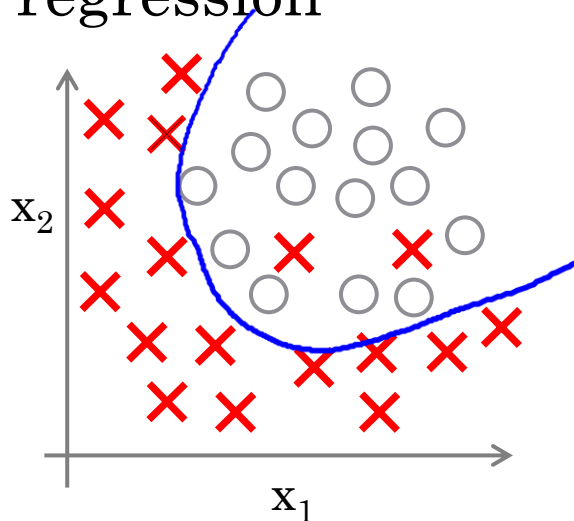
Example: Logistic regression



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

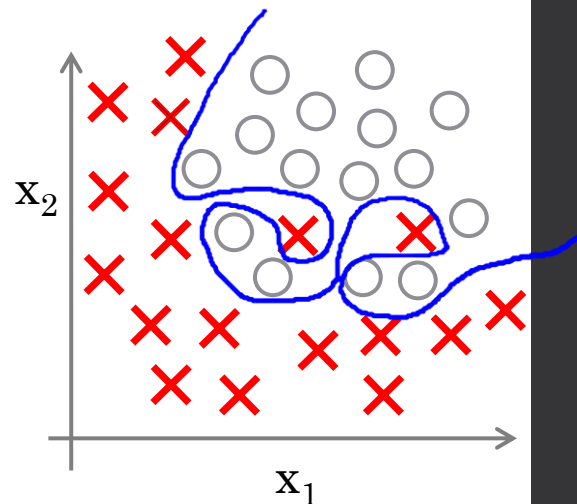
(g = sigmoid function)

"Underfit"



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

"Just right"

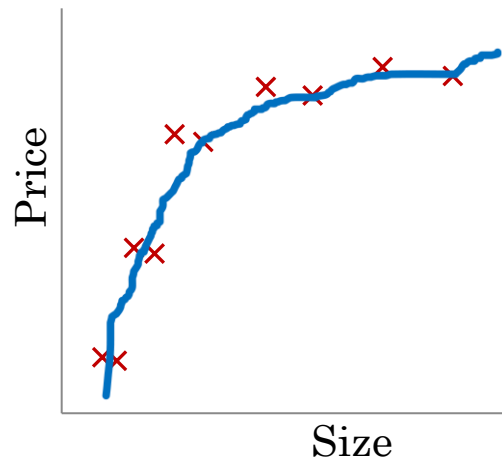
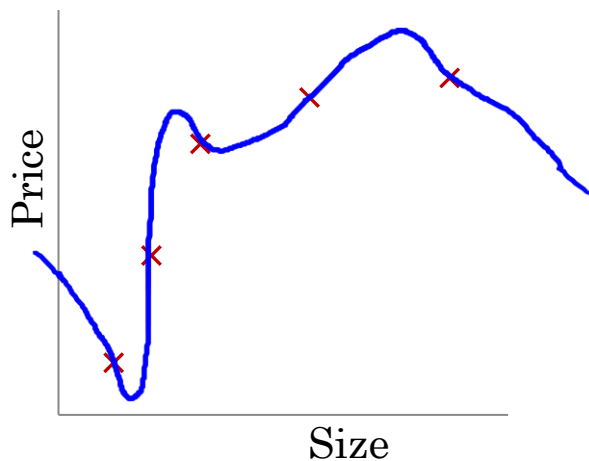


$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

"Overfit"

Addressing Overfitting:

1. The number one tool to use against overfitting to get more training data.
 - **Increase Training Data:** More diverse and representative data helps the model generalize better and reduces overfitting.



Addressing Overfitting:

2. A second option for addressing overfitting is to see if you can use fewer features.

x_1 = size of house

x_2 = no. of bedrooms

x_3 = no. of floors

x_4 = age of house

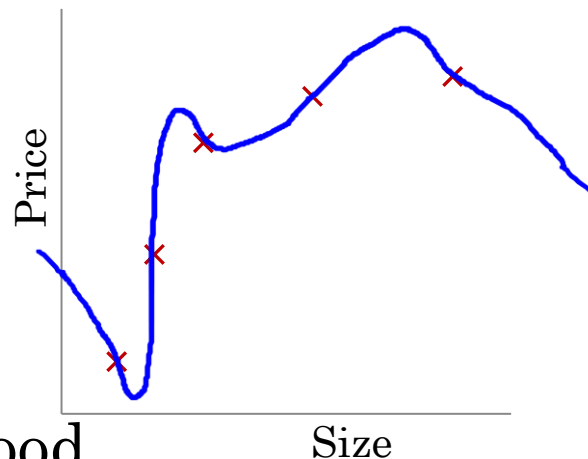
x_5 = average income in neighborhood

x_6 = kitchen size

$\vdots$

x_{100}

- Reduce number of features.
 - Manually select which features to keep.
 - Model selection algorithm.

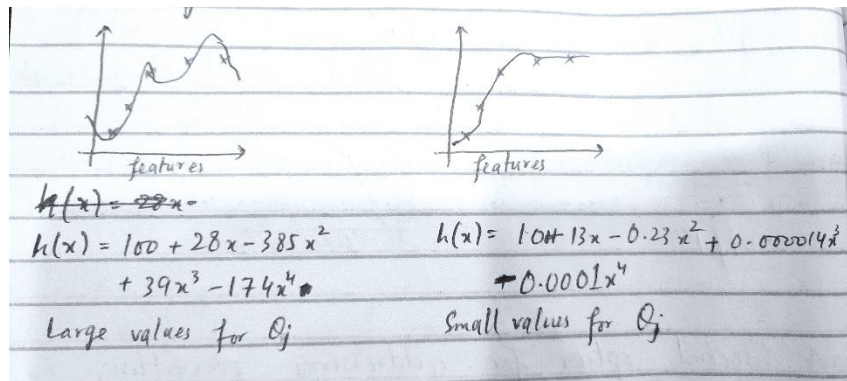


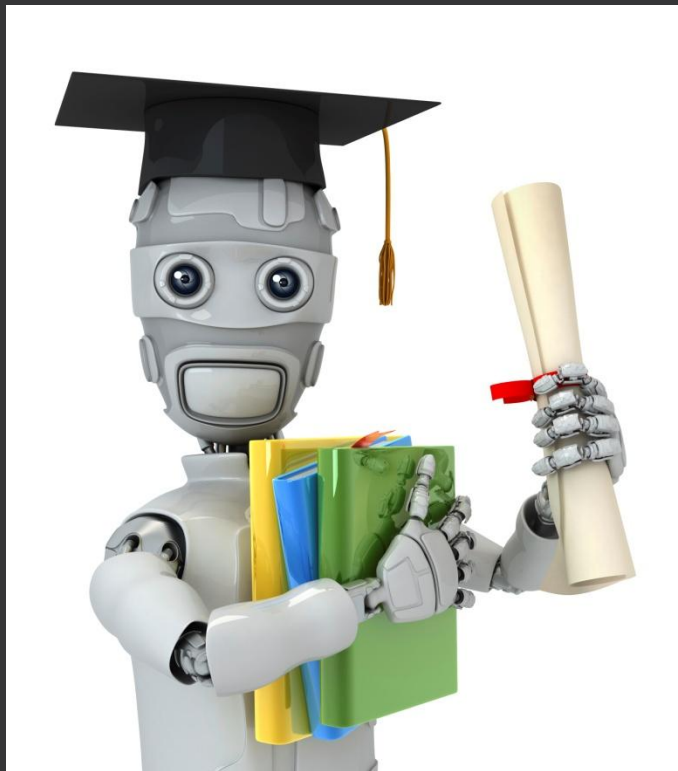
Addressing Overfitting:

3. The third option to reduce overfitting is to use regularization.

Regularization.

- Helps minimize overfitting and improves the generalization of the learning algorithm.
- Keeps all features but reduces the magnitude of model parameters θ_j (coefficients) to prevent excessive reliance on any single feature.
- Works well when there are many features, each contributing partially to predictions y .
- Provides a controlled way to reduce feature impact without completely eliminating them, unlike feature selection.



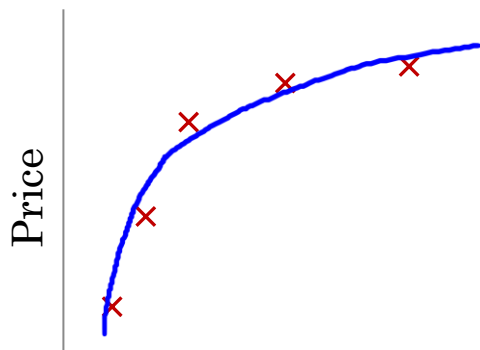


Machine Learning

Regularization

Cost Function

Intuition



Size of house

$$\theta_0 + \theta_1 x + \theta_2 x^2$$



Size of house

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

Suppose we penalize and make θ_3 , θ_4 really small.

$$\min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 1000 \underline{\theta_3^2} + 1000 \underline{\theta_4^2}$$

$$\theta_3 \approx 0$$

$$\theta_4 \approx 0$$

Regularization.

Small values for parameters $\theta_0, \theta_1, \dots, \theta_n$

- “Simpler” hypothesis
- Less prone to overfitting

Housing:

- Features: $x_1, x_2, \dots, x_{100}$
- Parameters: $\theta_0, \theta_1, \theta_2, \dots, \theta_{100}$

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

~~θ_0~~ $\theta_1, \theta_2, \theta_3, \dots, \theta_{100}$ ~~θ_0~~

Regularization.

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

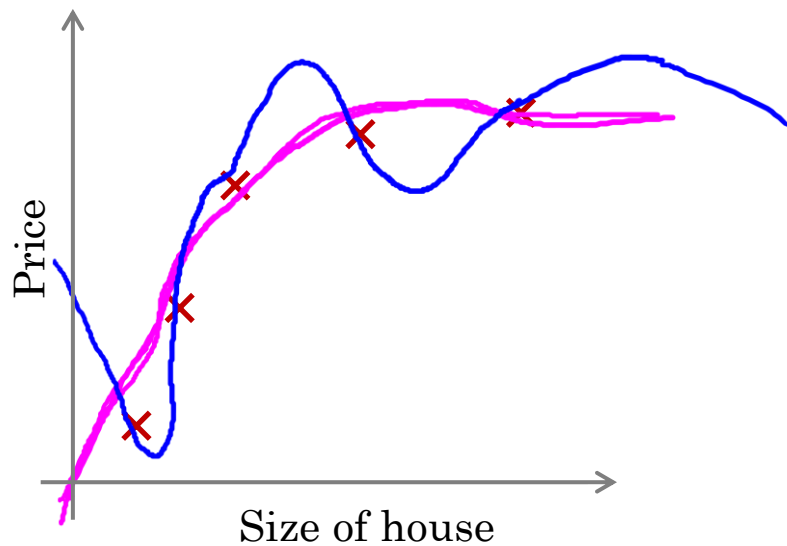
$$\min_{\theta} J(\theta)$$

$$\lambda \sum_{j=1}^n \theta_j^2 \rightarrow \text{Regularization term}$$

Add this regularization term at the end to shrink every single parameter, $\theta_1, \theta_2, \theta_3, \dots$

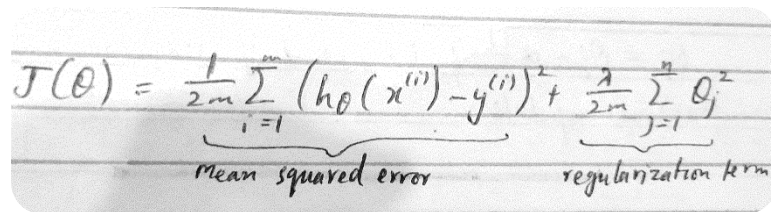
$\lambda \rightarrow$ “Lambda” Regularization parameter.

- λ is divide by $2m$ so that both the 1st and 2nd terms here are scaled $1/2m$. By scaling both terms the same way it becomes a little bit easier to choose a good value of λ .
- By convention we are not going to penalize the parameter θ_0 ,



Regularization.

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$



A handwritten version of the cost function $J(\theta)$ on lined paper. The formula is $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$. A bracket under the first summation is labeled "mean squared error", and a bracket under the second summation is labeled "regularization term".

This new cost function trades off two goals:

- **1st:** Fitting the training data well.
- **2nd:** Keeping the parameters small.

Impact of λ in Regularization

- **If $\lambda = 0$:** it means we are not using regularization term.
- **If λ is too big (enormous):** under fit.
 - The model applies excessive regularization, making it too simple and leading to underfitting.
- **If λ is in between:** Balance “just right”

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

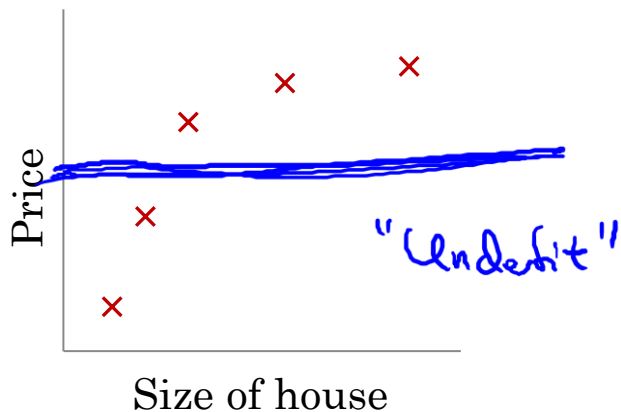
What if λ is set to an extremely large value (perhaps far too large for our problem, say $\lambda = 10^{10}$)?

- Algorithm works fine; setting λ to be very large can't hurt it.
- Algorithm fails to eliminate overfitting.
- Algorithm results in underfitting. (Fails to fit even training data well).
- Gradient descent will fail to converge.

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

What if λ is set to an extremely large value (perhaps far too large for our problem, say $\lambda = 10^{10}$)?



$h_{\theta}(x)$

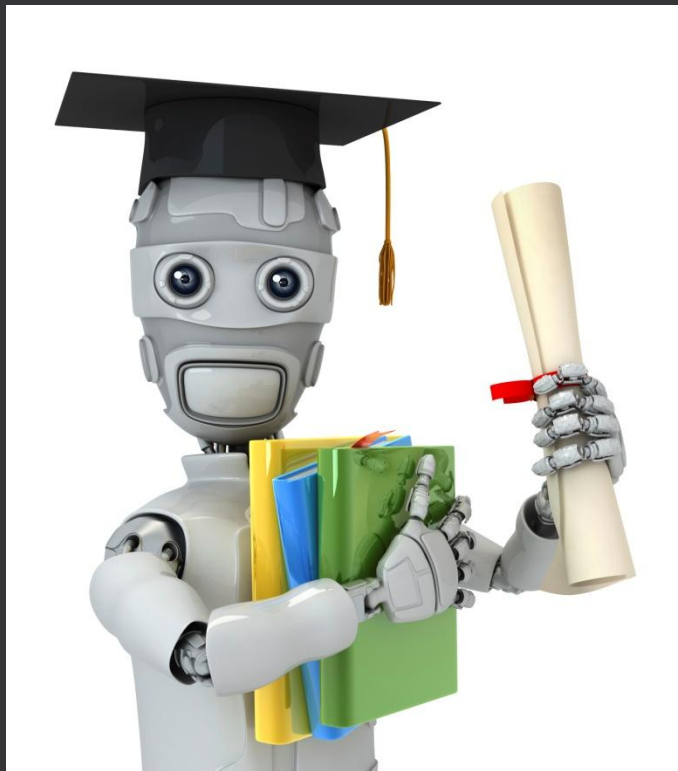
$$\theta_0 + \cancel{\theta_1 x} + \cancel{\theta_2 x^2} + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

$$\theta_1, \theta_2, \theta_3, \theta_4$$

$$\theta_1 \approx 0, \theta_2 \approx 0$$

$$\theta_3 \approx 0, \theta_4 \approx 0$$

$$h_{\theta}(x) = \theta_0$$



Machine Learning

Regularization

Regularized Linear Regression

Regularized linear regression

Cost Function: (regularized)

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$\min_{\theta} J(\theta)$$

Regularized linear regression

Gradient Descent:

$$\begin{array}{l} \text{repeat } \{ \\ \quad \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \\ \} \end{array} \quad \text{simultaneous update}$$

$$\begin{array}{l} \text{repeat } \{ \\ \quad \theta_0 := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta) \\ \quad \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \\ \} \end{array} \quad \begin{array}{l} \text{simultaneous update} \\ (j=1,2,\dots,n) \end{array}$$

Regularized linear regression

Gradient Descent:

$$\frac{\partial}{\partial \theta_0} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

→ don't have to penalize θ_0

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j$$

repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \quad x_0^{(i)} = 1$$
$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

$j = (1, 2, 3, \dots, n)$

} simultaneous update

Regularized linear regression

Gradient Descent:

Let's take a look for update rule θ_j and rewrite it in another way.

$$\theta_j := \theta_j - \alpha \left(\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \right) - \alpha \left(\frac{\lambda}{m} \theta_j \right)$$

$\rightarrow$ just rearranged the term

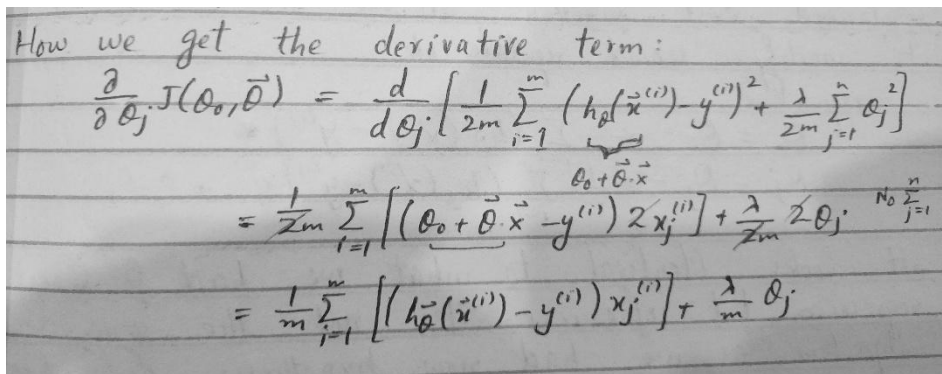
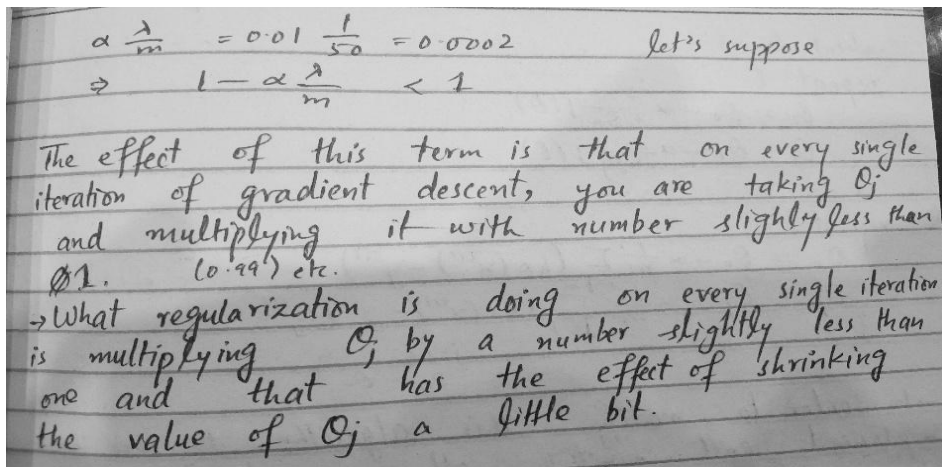
$$\theta_j := \theta_j - \alpha \frac{\lambda}{m} \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$
$$\theta_j := \theta_j \left(1 - \alpha \frac{\lambda}{m} \right) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

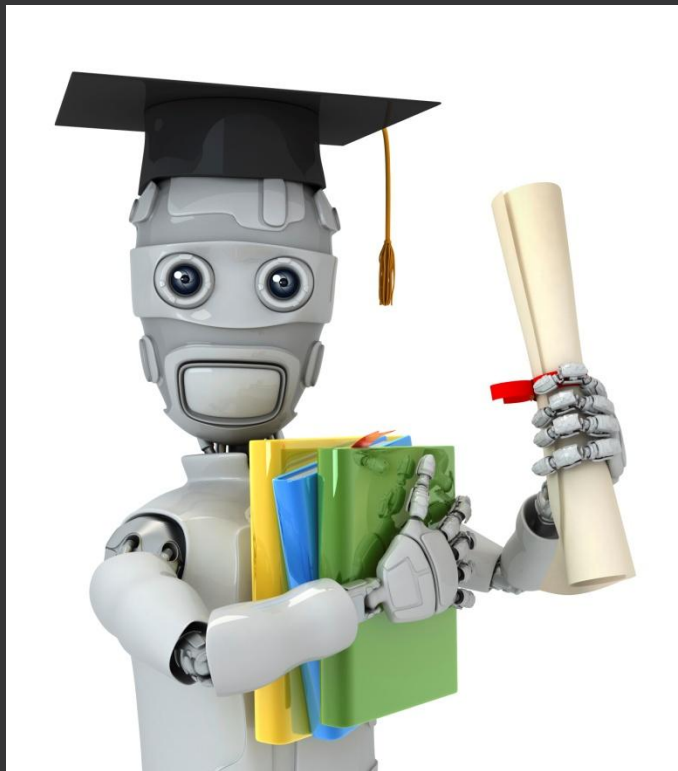
$\rightarrow$ number slightly less than 1 $\rightarrow$ usual gradient descent update for un-regularized Linear Regression

$$\theta_j := \theta_j \left(1 - \alpha \frac{\lambda}{m} \right) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Regularized linear regression

Gradient Descent:



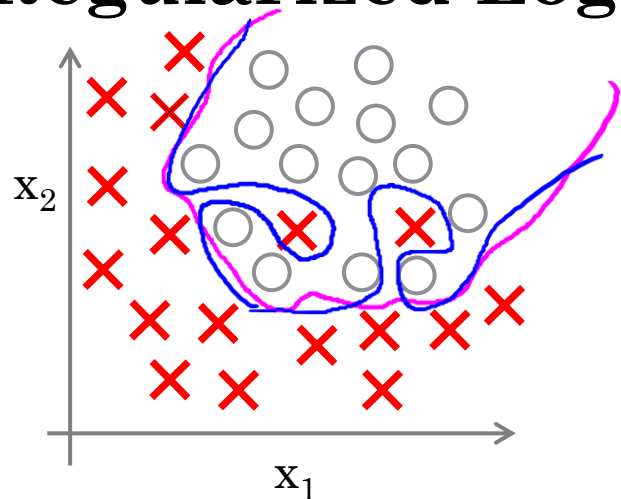


Machine Learning

Regularization

Regularized Logistic Regression

Regularized Logistic Regression.



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$

Cost function:

$$J(\theta) = - \left[\frac{1}{m} \sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

$\theta_1, \theta_2, \dots, \theta_n$

Gradient Descent

Repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} - \frac{1}{m} \theta_j \right] \leftarrow$$

$(j = \text{~~0~~, 1, 2, 3, \dots, n)$

$\theta_1, \dots, \theta_n$

$$\frac{\partial}{\partial \theta_j} J(\theta)$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Thank You